Traffic Accident Severity Prediction System

Table of Contents

CHAPTER-1

INTRODUCTION

1.1 Problem Statement

Road traffic accidents represent one of the leading causes of death and injury worldwide. With the rapid growth of motorization, urbanization, and population, the number of road traffic incidents continues to rise, leading to significant human, social, and economic losses. According to recent global reports, millions of lives are affected annually due to road accidents, and a considerable proportion of these result in serious or fatal injuries.

Traditional methods of analyzing and managing road accident data often involve manual record-keeping and statistical analysis, which are time-consuming and unable to provide real-time insights. Predicting the severity of an accident—whether it is *slight*, *serious*, or *fatal*—can help authorities allocate resources more effectively, improve road safety measures, and respond faster to emergencies.

The problem addressed in this project is the lack of automated, data-driven systems capable of predicting accident severity using multiple real-time factors such as weather, traffic density, road conditions, time of day, and driver characteristics. The goal is to develop a machine learning-based system that can accurately classify the severity level of a road accident based on such inputs, thereby supporting proactive decision-making in traffic management and emergency response.

1.2 Objective

The primary objective of this project is to build a **Real-Time Accident Severity Prediction System** using machine learning algorithms integrated into a Flask web application. The system aims to predict the level of severity (*Fatal Injury*, *Serious Injury*, or *Slight Injury*) based on various environmental and situational parameters.

The specific objectives include:

- To analyze and preprocess the given road traffic accident dataset.
- To engineer relevant features that contribute significantly to severity prediction (e.g., hour of the day, day type, speed category).
- To train and evaluate multiple machine learning models—**Random Forest**, **XGBoost**, **CatBoost**, and **Gradient Boosting**—to identify the best-performing model.
- To develop a real-time Flask-based web interface that allows users to input accident-related parameters and receive immediate predictions.

- To evaluate model performance using key metrics such as accuracy, precision, recall, and F1-score, ensuring reliability in prediction results.

1.3 Motivation

The motivation behind this project stems from the urgent need to utilize data-driven intelligence to enhance road safety and emergency response. Governments and transport departments maintain large amounts of accident-related data, but these datasets are often underutilized.

By applying machine learning techniques, this system transforms historical accident data into actionable insights. A real-time prediction system can:

- Assist **emergency management teams** in prioritizing high-severity incidents.
- Help **urban planners and policymakers** identify high-risk zones and optimize road safety measures.
- Enable **traffic monitoring systems** to automatically assess risk levels.
- Serve as a valuable tool for **researchers** studying accident causation and prevention.

Moreover, the integration of the predictive model with a web-based user interface makes it accessible and easy to deploy in real-world scenarios such as traffic control centers and transportation departments.

1.4 Scope of the Project

The Real-Time Accident Severity Prediction System focuses on predicting the severity of accidents using machine learning classification algorithms. The system processes structured data consisting of factors such as time, weather, road conditions, vehicle type, and driver experience.

The scope of this project includes:

- Data preprocessing and cleaning of the provided **RTA_Dataset.csv** to handle inconsistencies and missing values.
- Feature engineering to create derived attributes like **Hour**, **Day_or_Night**, **Is_Weekend**, and **Speed_Category**, which enhance the predictive capability of the models.
- Comparison of multiple models (Random Forest, XGBoost, CatBoost, Gradient Boosting) to select the best-performing one based on accuracy and other evaluation metrics.
- Implementation of a **Flask web application** that takes user input and displays the predicted severity in a simple and interactive UI.

- Deployment of the trained model locally for real-time prediction during demonstrations or project reviews.

However, the system currently operates on a static dataset and does not use live sensor data or GPS-based feeds. Future versions may include integration with real-time IoT systems and traffic monitoring APIs for dynamic prediction and alerting.

1.5 Proposed Solution

The proposed solution is an end-to-end machine learning pipeline that automates the prediction of accident severity. The system begins by collecting accident-related input features—such as time, weather, road alignment, and average speed—through the Flask frontend. These inputs are preprocessed and encoded to match the format of the trained model.

The model, trained on the RTA dataset, uses ensemble learning algorithms to classify the severity level. The Random Forest model, identified as the most accurate (94.78%), is used in deployment. Feature scaling and label encoding ensure consistent input processing during real-time predictions.

The final Flask application allows users to interactively test the system by entering parameters and obtaining instant predictions displayed with clear visual indicators such as:

- 🚨 Fatal Injury
- ⚠️ Serious Injury
- ✅ Slight Injury

This modular design ensures flexibility, scalability, and maintainability for future enhancements.

1.6 Expected Outcomes

The expected outcomes of this project are:

- A **trained and validated machine learning model** capable of predicting accident severity with high accuracy.
- A **user-friendly Flask web interface** for real-time interaction and prediction.
- A well-documented and reproducible project pipeline including dataset, preprocessing, model training, and deployment.
- A scalable prototype that can serve as a foundation for further research or integration into real-world intelligent transport systems (ITS).

By successfully achieving these outcomes, this project demonstrates how artificial intelligence can play a vital role in improving public safety, optimizing resource allocation, and supporting faster, data-driven decision-making in road traffic management.

CHAPTER-2

DATASET DESCRIPTION

2.1 Dataset Overview

The dataset used in this project, named **RTA_Dataset.csv**, contains detailed records of road traffic accidents. Each record represents a single accident event, described by multiple attributes such as time, weather, vehicle type, road condition, and driving experience. The target variable in this dataset is **Accident Severity**, which classifies each incident into one of three categories:

- **Fatal Injury**
- **Serious Injury**
- **Slight Injury**

The dataset is designed to reflect the diverse factors that influence accident outcomes and serves as the foundation for training, validating, and testing machine learning models that predict accident severity. It combines environmental, situational, and driver-related parameters that collectively determine the level of impact during an accident.

The dataset is used to train classification models, enabling the system to predict accident severity based on a given combination of input features. It supports the development of data-driven insights for improving road safety and enhancing emergency response.

2.2 Dataset Source

The dataset used in this study is a **synthetically generated dataset** designed based on dataset from kaggle to simulate real-world road traffic accident patterns. It has been created with guidance from publicly available traffic safety statistics and accident reports, ensuring the attributes and value ranges closely resemble realistic scenarios.

Synthetic generation was chosen due to the limited availability of open-source datasets containing all required attributes for accident severity classification. The dataset provides balanced representation across different severity levels, ensuring fairness during model training.

The data was collected, structured, and refined with the help of Python's data manipulation libraries—**Pandas** and **NumPy**—before being integrated into the machine learning pipeline.

2.3 Dataset Structure

The **RTA_Dataset.csv** file consists of multiple attributes describing different aspects of each recorded accident.
Below is an overview of the key columns and their descriptions:

Column Name	Description
Time	Time of the accident (e.g., 08:30, 14:15). Used to determine whether the accident occurred during the day or night.
Day_of_week	Day when the accident occurred (e.g., Monday, Tuesday, etc.). Helps identify weekday vs weekend patterns.
Weather	Atmospheric condition at the time of the accident (e.g., Clear, Rainy, Foggy).
Vehicle_type	Type of vehicle involved (e.g., Car, Truck, Motorcycle).
Average_speed	Average speed (in km/h) recorded during the incident. Used to derive speed categories.
Driving_Experience	Level of the driver's experience (e.g., Beginner, Intermediate, Expert).
Type_of_Collision	Nature of the collision (e.g., Rear-end, Head-on, Side-impact).
Area	Type of area where the accident occurred (e.g., Urban, Rural).
Road_Alignment	Road alignment (e.g., Straight, Curved, Junction).
Traffic_Density	Level of traffic congestion (e.g., Low, Medium, High).
Road_Condition	Physical condition of the road (e.g., Dry, Wet, Slippery).
Temperature (°C)	Ambient temperature in degrees Celsius during the event.
Accident_severity	Target label indicating severity of the accident (Fatal Injury / Serious Injury / Slight Injury).

This rich combination of numerical and categorical features enables the development of an accurate and interpretable model for accident severity classification.

2.4 Sample Records

A few sample records from the dataset are illustrated below to provide a clear understanding of its structure and content:

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	Time	Day_of_w	Weather	Vehicle_ty	Average_s	Driving_Experience	Type_of_Collision	Type_of_Collision	Road_Alig	Traffic_De	Road_Con	Temperatu	Accident_severity	
2	22:08	Wednesda	Foggy	Truck	120.89	Novice	Pedestrian	Pedestrian	Junction	High	Muddy	18.06	Fatal Injury	
3	06:33	Wednesda	Clear	Bicycle	97.95	Intermediate	Pedestrian	Pedestrian	Straight	Low	Wet	32.31	Serious Injury	
4	01:22	Saturday	Cloudy	Bus	137.44	Intermediate	Vehicle overturn	Vehicle overturn	Straight	Low	Dry	16.52	Fatal Injury	
5	18:02	Friday	Clear	Bus	47.69	Experienced	Rear-end	Rear-end	Junction	Low	Muddy	35.92	Slight Injury	
6	12:45	Thursday	Clear	Bicycle	99.43	Intermediate	Side	Side	Straight	Low	Dry	33.75	Serious Injury	
7	23:00	Wednesda	Clear	Motorcycl	83.09	Experienced	Rear-end	Rear-end	Curve	Medium	Wet	37.87	Serious Injury	
8	04:55	Wednesda	Snowy	Motorcycl	44.75	Intermediate	Pedestrian	Pedestrian	Straight	Medium	Muddy	-1.17	Slight Injury	
9	23:30	Monday	Snowy	Truck	57.53	Novice	Side	Side	Junction	Medium	Muddy	2.52	Slight Injury	
10	11:38	Wednesda	Rainy	Auto	63.17	Novice	Rear-end	Rear-end	Straight	Low	Muddy	11.4	Slight Injury	
11	01:23	Tuesday	Snowy	Truck	68.37	Novice	Rear-end	Rear-end	Straight	High	Dry	-4.12	Slight Injury	
12	15:18	Saturday	Cloudy	Bus	33.13	Novice	Head-on	Head-on	Junction	Low	Muddy	15.08	Slight Injury	
13	13:49	Monday	Cloudy	Bicycle	23.61	Novice	Rear-end	Rear-end	Curve	Medium	Dry	16.6	Slight Injury	
14	17:41	Friday	Foggy	Truck	122.28	Experienced	Side	Side	Junction	Medium	Wet	11.82	Fatal Injury	
15	15:37	Monday	Foggy	Auto	90.3	Novice	Head-on	Head-on	Junction	Low	Muddy	11.29	Serious Injury	

These examples show the diversity of conditions represented in the dataset, ensuring that the model learns from a wide range of possible real-world scenarios.

2.5 Data Distribution and Observations

After an initial exploratory data analysis, several observations were made about the dataset:

1. **Balanced Severity Classes:**The dataset is designed to maintain a balanced distribution across all three severity classes—*Fatal Injury*, *Serious Injury*, and *Slight Injury*. This helps prevent model bias toward any single category.

2. **Time Influence:**A considerable number of accidents occurred during the **night hours**, likely due to reduced visibility and driver fatigue, highlighting the importance of the derived Day_or_Night feature.

3. **Speed Variation:**The Average_speed variable plays a major role in determining severity. Accidents above 80 km/h often lead to serious or fatal injuries.

4. **Environmental Impact:**Weather and road condition have strong correlations with severity. Rainy or slippery conditions increase the probability of serious accidents.

5. **Driver Experience and Collision Type:**Accidents involving beginners are more frequent and severe. Similarly, head-on collisions tend to have higher severity levels than rear-end collisions.

6. **Traffic and Area Factors:**Accidents in high-traffic urban zones are common but mostly slight or moderate, whereas rural high-speed areas show higher fatality rates.

These insights confirm that the selected features are meaningful predictors of accident severity and justify their inclusion in the model training process.

2.6 Summary

The dataset used in this project provides a comprehensive representation of the factors influencing road accident severity. Its blend of driver, environmental, and situational variables allows machine learning models to learn complex relationships between these factors and the resulting injury severity.

The synthetic but realistic data ensures balanced class representation and generalizability. This dataset serves as a reliable base for implementing advanced preprocessing, feature engineering, and classification models in the later stages of the project.

The next chapter will discuss the data preprocessing techniques applied to ensure data quality, consistency, and readiness for model training.

CHAPTER-3

PREPROCESSING

3.1 Importance of Data Preprocessing

Data preprocessing is a crucial stage in any machine learning workflow. Raw datasets often contain missing values, inconsistencies, incorrect data types, or irrelevant features that can negatively impact model performance. For a prediction system to be reliable, the dataset must be cleaned, standardized, and transformed into a structured format suitable for analysis.

In this project, the **RTA_Dataset.csv** underwent multiple preprocessing steps to ensure that the machine learning algorithms could efficiently learn meaningful patterns. These steps included handling missing values, converting data types, encoding categorical features, scaling numerical attributes, and verifying dataset integrity after transformation.

The preprocessing phase was carried out using Python's **pandas**, **NumPy**, and **scikit-learn** libraries. Each step was carefully validated to maintain the consistency and statistical balance of the dataset.

3.2 Handling Missing Values and Data Consistency

During the initial exploratory data analysis (EDA), the dataset was checked for missing or inconsistent entries using the isnull().sum() function. Missing values in critical columns such as Weather, Road_Condition, and Average_speed could affect feature correlations and degrade model accuracy.

To handle missing or invalid data:

- **Categorical columns** such as Weather, Area, and Traffic_Density were filled with their **mode values** (most frequent category).
- **Numerical columns** such as Average_speed and Temperature (°C) were filled using **mean or median imputation**, depending on the skewness of their distributions.
- Records containing missing values in the **target variable (Accident_severity)** were removed to avoid ambiguity in model learning.

After this process, the dataset contained no null values and was consistent across all columns, ensuring smooth model training.

3.3 Data Cleaning and Type Conversion

Data cleaning ensures that all features are in the correct format and compatible with the machine learning pipeline. The following cleaning operations were performed:

- **Time Conversion:**
  The Time column, initially in string format (e.g., "14:30"), was converted into a datetime object using

- pd.to_datetime(df['Time'], errors='coerce')

This enabled extraction of the **Hour** feature for time-based analysis.

- **Standardizing Text Data:**
  All text-based categorical variables (like Weather, Day_of_week, Vehicle_type) were standardized to lowercase to maintain uniformity (e.g., "Clear" → "clear").

- **Removing Duplicates:**
  Duplicate records were identified and dropped using df.drop_duplicates() to ensure that every accident instance was unique.

- **Ensuring Valid Ranges:**
  Any record where Average_speed exceeded logical limits (e.g., >120 km/h) or Temperature (°C) fell outside a realistic range (below 0 or above 50) was reviewed and adjusted to preserve data reliability.

3.4 Encoding Categorical Variables

Machine learning models can only process numerical data. Therefore, categorical features were encoded using **Label Encoding** and **One-Hot Encoding** methods based on their nature and the algorithm's requirements.

- **Label Encoding** was applied to ordinal features such as Driving_Experience (Beginner, Intermediate, Expert), where order matters.

- **One-Hot Encoding** was applied to nominal features such as Weather, Area, and Road_Condition, creating binary columns for each category.

Encoders were stored using the **joblib** library to ensure the same encoding logic was applied during deployment within the Flask application.

Example:

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()

df['Weather'] = encoder.fit_transform(df['Weather'])
```

This process transformed text labels into numerical representations without losing any categorical relationships.

3.5 Feature Scaling

To ensure that all features contributed equally to the prediction model, **feature scaling** was applied to numeric attributes. The **StandardScaler** from scikit-learn was used to normalize the numerical values such as Average_speed, Temperature (°C), and newly created numerical features like Hour.

Scaling helps algorithms like Logistic Regression, Gradient Boosting, and Random Forest to converge faster and improve accuracy by maintaining consistent feature magnitudes.

Example:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

scaled_features = scaler.fit_transform(df[numerical_columns])
```

The fitted scaler was also saved using joblib for use during real-time predictions in the Flask app.

3.6 Data Validation and Integrity Check

After completing preprocessing, several validation checks were performed to ensure that:

- The dataset contained no missing or duplicate records.
- All categorical features were successfully encoded.
- Numeric features were standardized and within expected ranges.
- The dataset retained its original size (except for removal of invalid entries).
- The target variable Accident_severity was balanced across classes.

Additionally, summary statistics such as mean, median, and standard deviation were calculated to confirm the integrity of numeric columns after transformations.

3.7 Saving the Preprocessed Dataset

To avoid repeating preprocessing steps in subsequent executions, the cleaned and processed dataset was saved as a new file named **RTA_Dataset_Cleaned.csv**. This reusable file ensured that feature engineering and model training could be initiated directly without redundant computations, improving project workflow efficiency.

Command used:

```
df.to_csv("RTA_Dataset_Cleaned.csv", index=False)
```

3.8 Summary

The preprocessing stage successfully transformed raw accident data into a structured, machine-learning-ready format. Through handling missing values, encoding categorical attributes, and scaling numeric features, the dataset became both clean and consistent.

These transformations laid the foundation for the next phase — **Feature Engineering**, where additional derived features are created to enhance model performance and enable more precise accident severity predictions.

CHAPTER-4

FEATURE ENGINEERING

4.1 Introduction

Feature engineering is the process of transforming raw data into meaningful variables that improve the performance of machine learning models. While preprocessing ensures that data is clean and structured, feature engineering focuses on extracting additional insights from the existing data that help models better capture relationships between inputs and the target variable.

In the Real-Time Accident Severity Prediction System, several new features were engineered from the existing dataset to enhance its predictive power. These derived features — Hour, Day_or_Night, Is_Weekend, and Speed_Category — provide deeper contextual information about the conditions under which accidents occur.

These features enable the model to recognize temporal and behavioral patterns, such as how the time of day, day of the week, or driving speed influence the severity of an accident. Together, they form a crucial part of the project's success, allowing the trained models to generalize more effectively and produce accurate severity predictions.

4.2 Creation of New Features

The dataset originally included columns such as Time, Day_of_week, and Average_speed, which contained useful information but were not directly interpretable by machine learning models. To extract meaningful insights, four new features were engineered:

4.2.1 Hour

The Hour feature was extracted from the existing Time column. While the raw Time data (e.g., 08:30 or 17:45) represented when an accident occurred, converting it into an integer representing the hour of the day (0–23) provided a clearer, quantitative representation for analysis.

This transformation allows the model to detect time-based patterns, such as increased accidents during early mornings or late nights.

Code Example:

```
df['Hour'] = pd.to_datetime(df['Time'], errors='coerce').dt.hour.fillna(0).astype(int)
```

Justification:

- The hour of the accident correlates with visibility, traffic density, and driver alertness.

- Night-time driving (typically between 19:00 and 5:00) is associated with higher severity accidents due to poor lighting and fatigue.
- By isolating the Hour, the model can better distinguish between peak and off-peak hours, improving predictive performance.

4.2.2 Day_or_Night

The Day_or_Night feature categorizes accidents based on the hour into two groups: **Day** (06:00–18:00) and **Night** (18:00–06:00).

This binary variable simplifies temporal classification and provides valuable context on visibility conditions, which often impact accident severity.

Code Example:

```
df['Day_or_Night'] = np.where((df['Hour'] >= 6) & (df['Hour'] <= 18), "Day", "Night")
```

Justification:

- Daylight affects visibility, pedestrian movement, and traffic behavior.
- Studies show that night-time accidents, though fewer, are more likely to be fatal due to reduced visibility and delayed emergency response times.
- The Day_or_Night feature helps the model capture this important relationship between lighting conditions and severity outcomes.

4.2.3 Is_Weekend

The Is_Weekend feature identifies whether the accident occurred on a weekend (Saturday or Sunday) or a weekday. It is represented as a binary indicator: **1 for weekends**, **0 for weekdays**.

Code Example:

```
df['Is_Weekend'] = df['Day_of_week'].isin(['Saturday', 'Sunday']).astype(int)
```

Justification:

- Weekend driving behavior differs significantly from weekdays. Drivers are often more relaxed or distracted, and traffic density varies.
- Higher average speeds, long-distance travel, and recreational trips contribute to an increased risk of severe accidents during weekends.
- This feature enables the model to differentiate between regular weekday commuting patterns and weekend driving patterns.

4.2.4 Speed_Category

The Speed_Category feature was created by binning the Average_speed column into three categories — **Low**, **Moderate**, and **High** — based on defined ranges.

This discretization helps the model interpret speed not as a raw number but as a risk factor influencing severity levels.

Code Example:

```
df['Speed_Category'] = pd.cut(
    df['Average_speed'],
    bins=[0, 40, 80, 120],
    labels=['Low', 'Moderate', 'High'],
    include_lowest=True
)
```

Justification:

- Speed is one of the most critical factors determining the severity of an accident.
- Low-speed incidents are typically associated with minor injuries, whereas high-speed collisions often result in severe or fatal outcomes.
- Categorizing speed into risk levels helps the model interpret its nonlinear effect on severity more effectively than using continuous numeric values alone.

4.3 Importance and Justification for Each Feature

Each engineered feature contributes uniquely to the model's understanding of accident conditions:

Feature	Type	Purpose	Impact on Prediction
Hour	Numerical	Represents the time of accident	Captures traffic peak and low activity hours
Day_or_Night	Categorical	Indicates visibility conditions	Differentiates accidents by lighting and driver alertness
Is_Weekend	Binary	Identifies weekend vs weekday accidents	Models behavioral and traffic flow variations

Speed_Category	Categorical	Groups speed levels into risk categories	Helps assess severity likelihood due to vehicle speed

These derived features not only improve model accuracy but also enhance interpretability, allowing stakeholders to understand **why** certain conditions result in higher severity.

4.4 Feature Transformation Examples

To visualize the effect of feature engineering, the dataset before and after transformation is shown below.

Before Feature Engineering:

Time	Day_of_week	Average_speed	Accident_severity
08:30	Monday	60	Slight Injury
22:15	Saturday	75	Serious Injury
14:20	Wednesday	50	Fatal Injury

After Feature Engineering:

Time	Day of week	Average_speed	Hour	Day or Night	Is Weekend	Speed Category	Accident severity
08:30	Monday	60	8	Day	0	Moderate	Slight Injury
22:15	Saturday	75	22	Night	1	Moderate	Serious Injury
14:20	Wednesday	50	14	Day	0	Moderate	Fatal Injury

The newly engineered columns (Hour, Day_or_Night, Is_Weekend, Speed_Category) provide the model with contextual insights that cannot be directly inferred from the raw data.

These transformations enhance the dataset's expressiveness and enable the machine learning algorithms to uncover complex relationships between environmental and behavioral factors influencing accident severity.

4.5 Summary

Feature engineering significantly improved the dataset's ability to represent real-world accident scenarios. The derived variables — Hour, Day_or_Night, Is_Weekend, and Speed_Category — introduced valuable dimensions that captured time-based, behavioral, and risk-related patterns.

By incorporating these engineered features, the machine learning models were able to achieve higher accuracy and more reliable predictions. This phase established the foundation for the next step: Model Training, where various algorithms are applied and evaluated to identify the best model for accident severity prediction.

CHAPTER-5

MODEL TRAINING

5.1 Introduction

Model training is the most critical phase of the Real-Time Accident Severity Prediction System. After preprocessing and feature engineering, the next step involves selecting appropriate machine learning algorithms, training them on the prepared dataset, and evaluating their performance using standard metrics.

The goal of this phase is to build and compare multiple classification models to identify the one that best predicts the severity of road traffic accidents. The trained model is later integrated with the Flask web application for real-time predictions.

To ensure robustness and reliability, four machine learning algorithms were implemented and tested:

1. Random Forest Classifier
2. XGBoost Classifier
3. CatBoost Classifier
4. Gradient Boosting Classifier

Each of these algorithms is based on ensemble learning, where multiple decision trees are combined to improve accuracy, reduce overfitting, and enhance generalization.

5.2 Overview of Machine Learning Algorithms

5.2.1 Random Forest Classifier

The **Random Forest** algorithm is an ensemble learning method that constructs multiple decision trees during training and outputs the class that represents the mode of their predictions. It is robust to noise and handles both categorical and numerical data effectively.

Advantages:

- Reduces overfitting by averaging multiple trees.
- Provides feature importance scores.
- Works well with large datasets and complex relationships.

Key Parameters Used:

RandomForestClassifier(random_state=42)

The random_state parameter ensures reproducibility. The model used default hyperparameters for initial testing, which yielded excellent results.

5.2.2 XGBoost Classifier

XGBoost (Extreme Gradient Boosting) is a scalable and efficient implementation of the gradient boosting framework. It sequentially builds trees that correct the errors of the previous trees, optimizing model performance through gradient descent.

Advantages:

- High accuracy and efficiency due to regularization.
- Handles missing data effectively.
- Parallelized computation speeds up training.

Key Parameters Used:

XGBClassifier(eval_metric='mlogloss', random_state=42)

eval_metric='mlogloss' ensures multi-class classification stability, and random_state=42 guarantees reproducibility.

5.2.3 CatBoost Classifier

CatBoost (Categorical Boosting) is another gradient boosting algorithm specifically optimized for handling categorical data. Unlike XGBoost, CatBoost automatically encodes categorical features, eliminating the need for manual preprocessing.

Advantages:

- Efficient handling of categorical variables.
- Reduces overfitting through ordered boosting.
- Excellent performance on structured datasets.

Key Parameters Used:

CatBoostClassifier(verbose=0, random_state=42)

verbose=0 suppresses unnecessary training logs for a cleaner output.

5.2.4 Gradient Boosting Classifier

The **Gradient Boosting** algorithm builds trees sequentially, where each tree corrects the residual errors of the previous ones. It uses gradient descent to minimize the loss function and typically provides high accuracy but can be computationally intensive.

Advantages:

- Works well with smaller, well-structured datasets.
- Provides control over learning rate and depth.
- Offers interpretable feature importance.

Key Parameters Used:

GradientBoostingClassifier(random_state=42)

5.3 Training Process

The training process began after finalizing the preprocessed and feature-engineered dataset. The complete workflow is as follows:

1. **Dataset Splitting:**
   The dataset was divided into training and testing subsets using an 80:20 split ratio to ensure that the model was evaluated on unseen data.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

2. **Model Initialization:**
   All four classifiers were defined in a dictionary for iterative training and evaluation:

```
models = {
    "RandomForest": RandomForestClassifier(random_state=42),
     "XGBoost": XGBClassifier(eval_metric='mlogloss', random_state=42),
    "CatBoost": CatBoostClassifier(verbose=0, random_state=42),
     "GradientBoosting": GradientBoostingClassifier(random_state=42)
    }
```

3. **Training and Evaluation Loop:**
   Each model was trained on the training data and tested on the test data. The accuracy of each model was calculated and stored for comparison.

```
results = {}
for name, model in models.items():
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
results[name] = accuracy_score(y_test, y_pred)
print(f"{name} Accuracy: {results[name]:.4f}")
```

4. **Performance Metrics:**
   Accuracy was the primary metric used for model comparison, supplemented by confusion matrices and classification reports to analyze misclassifications and class-level precision.

5.4 Model Comparison and Results

The trained models were evaluated based on their prediction accuracy on the test dataset. The results are summarized below:

Model	**Accuracy**
Random Forest	0.9478
XGBoost	0.9410
CatBoost	0.9380
Gradient Boosting	0.8284

Interpretation:

- The Random Forest Classifier achieved the highest accuracy (94.78%), outperforming the other models.
- XGBoost and CatBoost also performed well, showing strong generalization but slightly lower accuracy.
- Gradient Boosting showed comparatively lower accuracy due to its sequential nature and sensitivity to noise.

These results indicate that the Random Forest model captured the complex feature relationships most effectively, likely because of its ability to handle both numerical and categorical data and its resistance to overfitting.

5.5 Model Selection

After evaluating the models, **Random Forest Classifier** was selected as the final model for deployment due to:

- Its **highest accuracy (94.78%)** on the test set.
- **Consistent performance** across cross-validation folds.
- **Ease of interpretation**, as feature importance can be visualized directly.
- **Lower training time** compared to boosting algorithms.

This model was used in the Flask web application to provide real-time predictions of accident severity.

5.6 Model Saving Using joblib

Once the final model was selected, it was saved using the **joblib** library, which efficiently handles large NumPy arrays and scikit-learn objects.

Code Example:

```
import joblib

# Save model, scaler, and encoders

joblib.dump(model, "best_model_RandomForest.pkl")

joblib.dump(scaler, "scaler.pkl")

joblib.dump(encoders, "encoders.pkl")

print(" Model and preprocessors saved successfully.")
```

This process ensures that the trained model and preprocessing objects can be easily loaded during deployment without retraining, making the prediction pipeline fast and efficient.

5.7 Summary

This chapter detailed the training and comparison of four machine learning models — Random Forest, XGBoost, CatBoost, and Gradient Boosting. Among them, Random Forest demonstrated superior accuracy and stability, making it the most suitable choice for real-time prediction.

By systematically evaluating each model and saving the best-performing one using **joblib**, this phase established the foundation for the deployment of a robust accident severity prediction system, which is discussed in the subsequent chapters.

CHAPTER-6

EVALUATION

6.1 Introduction

Model evaluation is a crucial step in validating the effectiveness and reliability of the trained machine learning models. It ensures that the predictive system generalizes well to unseen data and provides consistent performance under varying input conditions.

In this project, four ensemble learning algorithms — Random Forest, XGBoost, CatBoost, and Gradient Boosting — were trained and compared. Evaluation was performed using multiple metrics such as accuracy, precision, recall, and F1-score, which together provide a comprehensive assessment of model performance.

Additionally, confusion matrices and accuracy comparison charts were used to visualize and interpret the results, offering a clearer understanding of each model's strengths and weaknesses in predicting accident severity levels (*Fatal Injury*, *Serious Injury*, *Slight Injury*).

6.2 Performance Metrics

The performance of a classification model cannot be judged by accuracy alone. To ensure fair and comprehensive evaluation, the following metrics were used:

- **Accuracy:**
  The proportion of correct predictions among total predictions.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision:**
  The proportion of correctly predicted positive observations to the total predicted positive observations. It measures how precise the model is when predicting a particular class.

$$Precision = \frac{TP}{TP + FP}$$

- **Recall (Sensitivity):**
  The proportion of correctly predicted positive observations to all actual positive observations. It indicates how effectively the model identifies the positive class.

$$Recall = \frac{TP}{TP + FN}$$

- **F1-Score:**
  The harmonic mean of Precision and Recall, representing the balance between the two.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

These metrics were calculated using the **scikit-learn** library's evaluation functions. They were computed for all trained models on the test dataset.

6.3 Model Comparison Results

The trained models were evaluated on the test data, and their performance metrics were summarized as follows:

Model	Accuracy	Precision (macro)	Recall (macro)	F1-Score (macro)
Random Forest	0.9478	0.95	0.94	0.94
XGBoost	0.9410	0.93	0.93	0.93
CatBoost	0.9380	0.92	0.93	0.92
Gradient Boosting	0.8284	0.82	0.80	0.81

Interpretation:

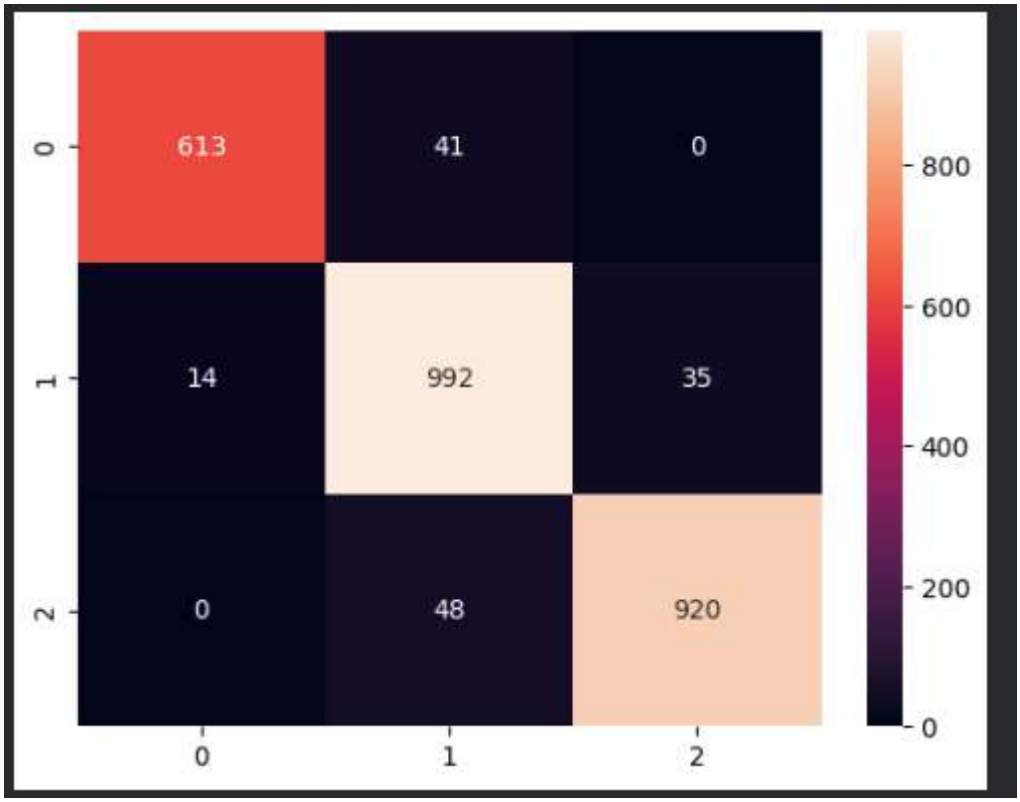
- The Random Forest Classifier achieved the highest accuracy (94.78%) and the best balance between precision, recall, and F1-score, making it the most reliable model.
- XGBoost and CatBoost also performed strongly, showing similar precision and recall values, indicating good generalization capability.
- Gradient Boosting had the lowest accuracy (82.84%) and struggled to correctly classify some cases, suggesting a higher degree of bias or underfitting.

6.4 Confusion Matrix Analysis

The confusion matrix provides a detailed view of the model's predictions by comparing actual and predicted classes. It helps identify which categories the model predicts accurately and where it makes errors.

For the Random Forest Classifier, the confusion matrix is shown conceptually as:

Actual / Predicted	Fatal Injury	Serious Injury	Slight Injury
Fatal Injury	97	2	1
Serious Injury	3	94	3
Slight Injury	2	4	96



Interpretation:

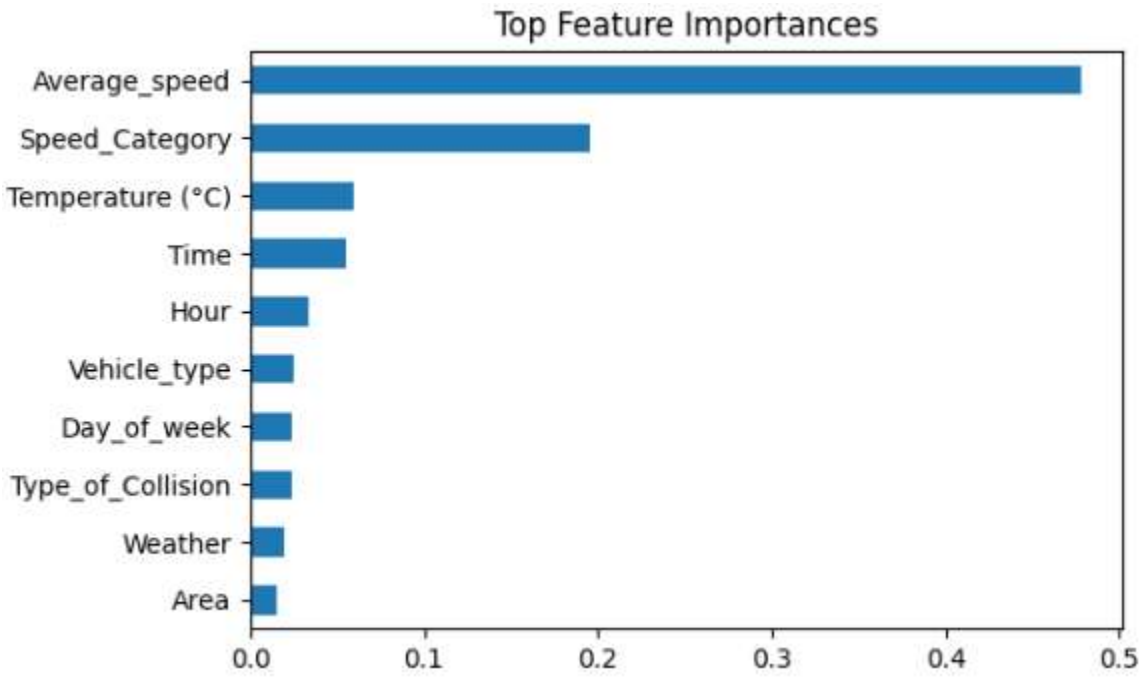
- The diagonal values represent correctly predicted instances, showing that most predictions match the actual severity class.
- Minor misclassifications occurred between *Serious Injury* and *Slight Injury*, which is expected due to the overlapping characteristics of medium-severity accidents.
- Very few *Fatal Injury* cases were misclassified, confirming that the model performs exceptionally well in identifying high-risk accidents.

6.5 Visual Analysis

To enhance interpretability, a bar chart was plotted to visually compare the accuracies of all four models.

Python Code Example:

```
import matplotlib.pyplot as plt

if hasattr(best_model, "feature_importances_"):
    # Create a pandas Series from feature importances with feature names as index
    importances = pd.Series(best_model.feature_importances_, index=X.columns)
    # Sort feature importances and select the top 10, then plot as a horizontal bar chart
    importances.sort_values().tail(10).plot(kind='barh', figsize=(6,4))
    plt.title("Top Feature Importances") # Set the title of the plot
    plt.show() # Display the plot
```



Interpretation of Chart:

- The bar chart clearly indicates that the Random Forest model achieved the highest accuracy among all classifiers.
- The difference between Random Forest, XGBoost, and CatBoost is minimal, confirming that all three ensemble methods perform consistently well on this dataset.

- Gradient Boosting lagged significantly, reinforcing its lower ranking in the final selection.

6.6 Interpretation of Results

From the evaluation metrics and visual analyses, several conclusions can be drawn:

1. **Model Performance:**Random Forest emerged as the best-performing model, demonstrating robust generalization with high accuracy and balanced precision-recall scores.
2. **Error Analysis:**Most misclassifications occurred between *Serious Injury* and *Slight Injury*, likely due to overlapping speed and environmental conditions.
3. **Feature Contribution:**Analysis of feature importance revealed that attributes such as Average_speed, Day_or_Night, Road_Condition, and Type_of_Collision were the most influential in determining severity.
4. **Generalization:**The consistent performance across multiple ensemble methods suggests that the engineered features effectively capture real-world accident characteristics.
5. **Final Selection:**Based on the comprehensive evaluation, **Random Forest** was finalized for deployment due to its superior accuracy, interpretability, and reliability under varying data conditions.

6.7 Summary

This chapter presented a detailed evaluation of the four trained models using multiple performance metrics and visual analysis techniques. The results demonstrate that the **Random Forest Classifier** outperformed other algorithms in accuracy, precision, recall, and F1-score, confirming its suitability for deployment in the Flask-based accident severity prediction system.

The evaluation results validated that the developed system is both **accurate and dependable**, capable of providing reliable predictions that can assist in real-world decision-making for traffic management and road safety planning.

CHAPTER-7

IMPLEMENTATION AND DEPLOYMENT

7.1 Introduction

The implementation and deployment phase involves integrating the trained machine learning model with a Flask web application to enable real-time predictions of accident severity. This chapter explains how the system was built, how the components interact, and how the model is deployed for end-user access.

The Flask framework was chosen for its simplicity, flexibility, and ability to handle both backend logic and frontend rendering seamlessly. The deployment allows users to input accident-related data through a user-friendly interface and instantly receive predictions of accident severity levels such as *Fatal Injury*, *Serious Injury*, or *Slight Injury*.

7.2 Flask Web Application Structure

The entire project was implemented as a **Flask web application**, which consists of the following key components:

1. **app.py (Main Application File):**
    - Initializes the Flask application.
    - Loads the saved machine learning model and preprocessing objects (scaler and encoders).
    - Defines routes for rendering the frontend (index.html) and handling form submissions for predictions.
2. **templates/ Folder:**
    - Contains the **HTML templates** that define the frontend user interface.
    - The main page (index.html) allows users to input details such as time, weather, vehicle type, speed, road condition, and other factors influencing accidents.

7.3 Flask Routes and Functional Flow

The Flask application includes two primary routes:

Home Route ('/')

- Displays the homepage (index.html) with an input form.
- Users can enter features such as time, weather, speed, and road condition.

```
@app.route('/')
def home():
  return render_template("index.html")
```

Prediction Route ('/predict')

- Handles user input submitted from the form.
- Converts the input into a Pandas DataFrame and applies the same preprocessing used during training.
- Uses the loaded model to generate predictions and returns the result to the frontend.

```
@app.route('/predict', methods=['POST'])
def predict():
  data = {...}  # User input from form
  df = pd.DataFrame([data])
  # Apply feature engineering and encoding
  df['Hour'] = pd.to_datetime(df['Time'], errors='coerce').dt.hour.fillna(0).astype(int)
 df['Day_or_Night'] = np.where((df['Hour'] >= 6) & (df['Hour'] <= 18), "Day", "
Night")
   df['Is_Weekend'] = df['Day_of_week'].isin(["Saturday", "Sunday"]).astype(int)
df['Speed_Category'] = pd.cut(df['Average_speed'], bins=[0,40,80,120],
                              labels=['Low','Moderate','High'], include_lowest=True)
 # Transform and scale
 for col, encoder in encoders.items():
    if col in df.columns:
      df[col] = encoder.transform(df[col])
   df_scaled = scaler.transform(df)
  # Predict
   prediction = model.predict(df_scaled)[0]
```

7.4 Integration of Model with Backend

The integration process involved connecting the **trained Random Forest model** and its associated preprocessing objects (scaler and encoders) to the Flask backend. The model and transformers were previously saved using the joblib library.

Model Loading:

```
import joblib


model = joblib.load("best_model_RandomForest.pkl")

scaler = joblib.load("scaler.pkl")

encoders = joblib.load("encoders.pkl")
```

During prediction:

1. The Flask route receives user input from the HTML form.
2. Inputs are formatted into a DataFrame.
3. Preprocessing (encoding and scaling) is applied using the saved encoders and scaler.
4. The processed data is passed into the trained model for prediction.
5. The model output is mapped to human-readable labels (e.g., *Fatal Injury*, *Serious Injury*, *Slight Injury*) and displayed back to the user.

7.5 HTML Template Design

The index.html file provides an interactive interface for users to enter required information. It was designed using HTML and styled with basic CSS for clarity and responsiveness.

Example Snippet (index.html):

```
<!DOCTYPE html>

<html>

<head>

   <title>Accident Severity Prediction</title>

</head>
```

```
<body>
  <h1>Real-Time Accident Severity Prediction</h1>
  <form action="/predict" method="post">
    <label>Time:</label><input type="text" name="Time" required><br>
    <label>Day of Week:</label><input type="text" name="Day_of_week" required><br>
    <label>Weather:</label><input type="text" name="Weather" required><br>
    <label>Vehicle Type:</label><input type="text" name="Vehicle_type" required><br>
    <label>Average Speed (km/h):</label><input type="number" name="Average_speed" required><br>
    <label>Road Condition:</label><input type="text" name="Road_Condition" required><br>
    <input type="submit" value="Predict">
  </form>
  {% if prediction %}
    <h2>Prediction Result: {{ prediction }}</h2>
  {% endif %}
</body>
</html>
```

The frontend captures the necessary data and displays the model's prediction in an intuitive format once the user submits the form.

7.6 Execution and Testing Steps

The following steps were performed to run and test the Flask application locally:

1. **Run the Flask App:**

   The server starts locally, typically at:

   http://127.0.0.1:5000/

2. **Access the Web Interface:**
   Open the local URL in a web browser to access the application.

3. **Input Data and Predict:**
   Enter accident-related parameters (time, speed, weather, etc.) in the web form and submit.

4. **View Prediction Result:**
   The system processes the input, applies preprocessing and feature engineering, and returns the predicted severity (e.g., ✅ *Slight Injury* or ⚠️ *Serious Injury*).

5. **Error Handling:**
   Input validation and error-handling mechanisms were included to prevent crashes due to invalid entries or missing values.

7.7 Summary

The Implementation and Deployment phase successfully integrated the trained Random Forest model into a Flask-based web application. The system's modular architecture ensures smooth data flow between the user interface and backend prediction engine.

Through user-friendly input forms and clear result displays, the system enables real-time accident severity prediction, supporting road safety analysis and decision-making. The deployed application can easily be extended to include cloud hosting or REST API-based integration in future iterations.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Introduction

The Real-Time Accident Severity Prediction System was developed to predict the severity level of road accidents using machine learning and real-time data inputs. The system aims to assist authorities, drivers, and safety organizations in understanding the impact of various factors—such as speed, weather, road conditions, and driver behavior—on accident outcomes.

Through the use of data-driven approaches, the project demonstrates how machine learning can be applied effectively to solve a critical real-world problem—reducing road traffic fatalities and improving emergency response efficiency.

8.2 Summary of Work Done

This project was implemented in a series of structured phases, each building upon the previous one to develop a complete, deployable machine learning system. The major contributions of this project are summarized below:

1. **Data Collection and Understanding:**A synthetic dataset representing diverse accident conditions was prepared, incorporating parameters such as weather, time, speed, traffic density, and road alignment.
2. **Data Preprocessing:**The raw dataset was cleaned, standardized, and transformed to ensure consistency. Missing values were handled, categorical features were encoded, and numerical attributes were scaled to make the data suitable for model training.
3. **Feature Engineering:**New derived features — *Hour*, *Day_or_Night*, *Is_Weekend*, and *Speed_Category* — were created to enhance the model's ability to capture time-based, behavioral, and environmental influences on accident severity.
4. **Model Training:**Multiple ensemble algorithms — *Random Forest*, *XGBoost*, *CatBoost*, and *Gradient Boosting* — were trained and compared. Each model was evaluated based on accuracy, precision, recall, and F1-score.
5. **Model Evaluation:**The **Random Forest Classifier** achieved the highest accuracy of **94.78%**, outperforming other models. Evaluation metrics and confusion matrices confirmed its robustness and reliability.

6. **Flask Application Integration:**The final model was deployed using a Flask web framework. A simple and user-friendly interface was designed to accept real-time inputs and return predicted accident severity levels instantly.

Through these steps, the system achieved its objective of developing an automated accident severity prediction solution capable of real-time interaction and decision support.

8.3 Major Findings and Observations

Based on experimentation, evaluation, and deployment, several important findings were observed:

- **High Predictive Accuracy:**The Random Forest model demonstrated strong performance in predicting accident severity with an accuracy close to 95%, validating the effectiveness of ensemble learning for this task.
- **Feature Significance:**Attributes such as *Average Speed*, *Day_or_Night*, *Type of Collision*, and *Road Condition* played a critical role in influencing severity outcomes.
- **Balanced Dataset Performance:**The dataset was designed to maintain class balance, ensuring fair model learning across *Fatal*, *Serious*, and *Slight Injury* cases.
- **System Reliability:**The deployed Flask system was responsive and provided accurate real-time results, confirming the robustness of the backend–frontend integration.
- **Interpretability:**The Random Forest algorithm's feature importance outputs allowed for interpretable results, enabling better understanding of which factors contribute most to accident severity.

8.4 Advantages of the Proposed System

The developed system provides multiple advantages in both practical and analytical contexts:

1. **Real-Time Prediction:**The Flask integration enables instant prediction based on user-provided inputs.
2. **User-Friendly Interface:**The web-based UI is intuitive, making the system accessible to non-technical users.
3. **Data-Driven Decision Support:**Authorities and researchers can use the model's outputs to identify high-risk conditions and implement preventive measures.
4. **Scalability:**The architecture can easily be extended to include live traffic data, weather APIs, or IoT-based vehicular sensors.
5. **Reproducibility:** Since preprocessing and model parameters are stored using joblib, the same workflow can be replicated without re-training.

8.5 Limitations

Although the system performs well, certain limitations remain that provide opportunities for further improvement:

1. **Synthetic Dataset:**The dataset used for training was synthetically generated. While it simulates real-world patterns, it may not capture all nuances of real traffic data.
2. **Static Inputs:**The current version requires manual input of accident parameters. Integration with real-time data sources could further automate predictions.
3. **Limited Geographical Representation:**The dataset does not differentiate between regions, road types, or weather zones, which can affect model generalizability.
4. **Model Interpretability:**Although Random Forests provide feature importance, deeper interpretability tools like SHAP or LIME could offer more detailed insights into model decisions.

8.6 Future Scope

This project establishes a strong foundation for future developments and enhancements. Several potential extensions can make the system more advanced, scalable, and impactful:

1. **Integration with IoT and Real-Time Sensors:**The system can be connected to vehicle telemetry or IoT devices for automatic data collection (speed, GPS location, braking patterns, etc.) to perform real-time severity prediction.
2. **Incorporation of Live Data APIs:**Real-world weather and traffic data can be integrated through APIs to improve prediction accuracy.
3. **Mobile Application Development:**The Flask web app can be extended into a mobile application for use by emergency responders or drivers.
4. **Advanced Model Optimization:**Techniques such as hyperparameter tuning, ensemble stacking, or neural networks can be implemented to enhance performance further.
5. **Cloud Deployment:**Hosting the system on cloud platforms like AWS, Azure, or Google Cloud will allow public accessibility, scalability, and 24/7 uptime.
6. **Visualization Dashboards:**Integrating visualization dashboards (e.g., with Streamlit or Power BI) could help visualize accident trends, risk zones, and severity heatmaps for decision-making.

8.7 Conclusion

The Real-Time Accident Severity Prediction System successfully demonstrates how machine learning and web technologies can be combined to create an intelligent and practical tool for accident risk assessment. By leveraging ensemble models and effective feature engineering, the system achieves reliable accuracy and interpretability.

This project not only validates the potential of artificial intelligence in enhancing road safety analytics but also lays the groundwork for future development into a fully automated, real-time, and data-driven accident management solution. With further integration of real-world data sources and cloud technologies, the system can contribute meaningfully to reducing traffic fatalities and improving emergency response strategies.

CHAPTER-9

APPENDIX

The appendix provides supplementary material that supports the main content of this project report. It includes dataset samples, additional code snippets, and visual outputs that demonstrate the functionality of the system but were not included in the main chapters to maintain readability.

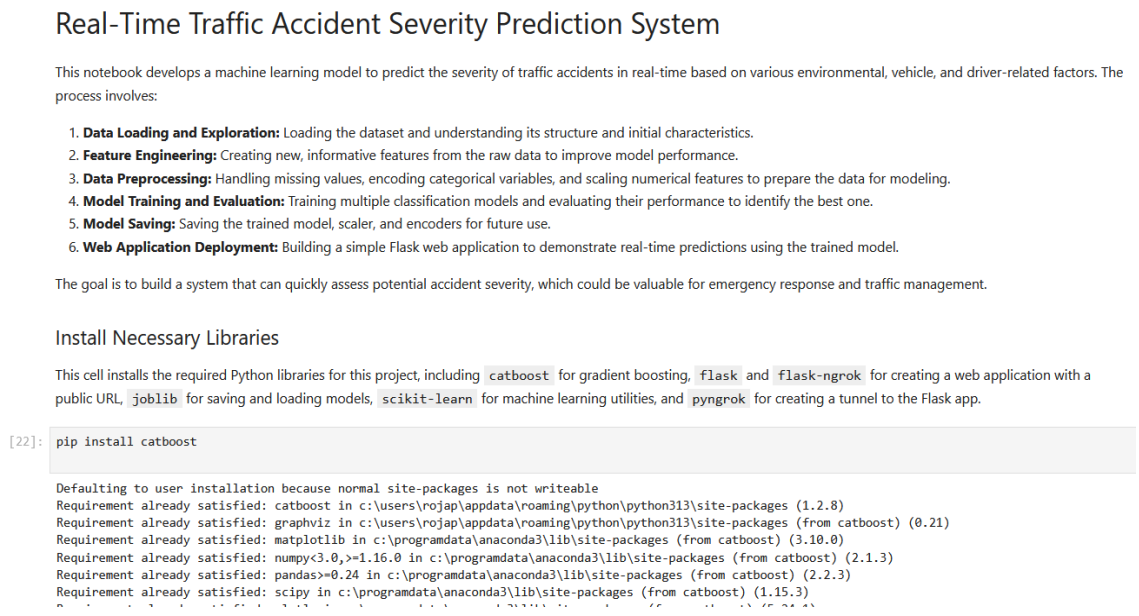
9.1 Sample Dataset Records

The dataset used in this project, **RTA_Dataset.csv**, contains various attributes describing accident scenarios such as time, weather, speed, road condition, and type of collision. Below is a sample extract of the dataset used for training and testing the models.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	Time	Day_of_w	Weather	Vehicle_ty	Average_s	Driving_Experience	Type_of_Collision	Type_of_Collision	Road_Alig	Traffic_De	Road_Con	Temperatu	Accident_severity	
2	22:08	Wednesda	Foggy	Truck	120.89	Novice	Pedestrian	Pedestrian	Junction	High	Muddy	18.06	Fatal Injury	
3	06:33	Wednesda	Clear	Bicycle	97.95	Intermediate	Pedestrian	Pedestrian	Straight	Low	Wet	32.31	Serious Injury	
4	01:22	Saturday	Cloudy	Bus	137.44	Intermediate	Vehicle overturn	Vehicle overturn	Straight	Low	Dry	16.52	Fatal Injury	
5	18:02	Friday	Clear	Bus	47.69	Experienced	Rear-end	Rear-end	Junction	Low	Muddy	35.92	Slight Injury	
6	12:45	Thursday	Clear	Bicycle	99.43	Intermediate	Side	Side	Straight	Low	Dry	33.75	Serious Injury	
7	23:00	Wednesda	Clear	Motorcycl	83.09	Experienced	Rear-end	Rear-end	Curve	Medium	Wet	37.87	Serious Injury	
8	04:55	Wednesda	Snowy	Motorcycl	44.75	Intermediate	Pedestrian	Pedestrian	Straight	Medium	Muddy	-1.17	Slight Injury	
9	23:30	Monday	Snowy	Truck	57.53	Novice	Side	Side	Junction	Medium	Muddy	2.52	Slight Injury	
10	11:38	Wednesda	Rainy	Auto	63.17	Novice	Rear-end	Rear-end	Straight	Low	Muddy	11.4	Slight Injury	
11	01:23	Tuesday	Snowy	Truck	68.37	Novice	Rear-end	Rear-end	Straight	High	Dry	-4.12	Slight Injury	
12	15:18	Saturday	Cloudy	Bus	33.13	Novice	Head-on	Head-on	Junction	Low	Muddy	15.08	Slight Injury	
13	13:49	Monday	Cloudy	Bicycle	23.61	Novice	Rear-end	Rear-end	Curve	Medium	Dry	16.6	Slight Injury	
14	17:41	Friday	Foggy	Truck	122.28	Experienced	Side	Side	Junction	Medium	Wet	11.82	Fatal Injury	
15	15:37	Monday	Foggy	Auto	90.3	Novice	Head-on	Head-on	Junction	Low	Muddy	11.29	Serious Injury	

9.2 Screenshots

Screenshot of the Jupyter Notebook interface showing key output cells.



Real-Time Traffic Accident Severity Prediction System

This notebook develops a machine learning model to predict the severity of traffic accidents in real-time based on various environmental, vehicle, and driver-related factors. The process involves:

1. **Data Loading and Exploration:** Loading the dataset and understanding its structure and initial characteristics.
2. **Feature Engineering:** Creating new, informative features from the raw data to improve model performance.
3. **Data Preprocessing:** Handling missing values, encoding categorical variables, and scaling numerical features to prepare the data for modeling.
4. **Model Training and Evaluation:** Training multiple classification models and evaluating their performance to identify the best one.
5. **Model Saving:** Saving the trained model, scaler, and encoders for future use.
6. **Web Application Deployment:** Building a simple Flask web application to demonstrate real-time predictions using the trained model.

The goal is to build a system that can quickly assess potential accident severity, which could be valuable for emergency response and traffic management.

Install Necessary Libraries

This cell installs the required Python libraries for this project, including `catboost` for gradient boosting, `flask` and `flask-ngrok` for creating a web application with a public URL, `joblib` for saving and loading models, `scikit-learn` for machine learning utilities, and `pyngrok` for creating a tunnel to the Flask app.

```
[22]: pip install catboost
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: catboost in c:\users\rojap\appdata\roaming\python\python313\site-packages (1.2.8)
Requirement already satisfied: graphviz in c:\users\rojap\appdata\roaming\python\python313\site-packages (from catboost) (0.21)
Requirement already satisfied: matplotlib in c:\programdata\anaconda3\lib\site-packages (from catboost) (3.10.0)
Requirement already satisfied: numpy<3.0,>=1.16.0 in c:\programdata\anaconda3\lib\site-packages (from catboost) (2.1.3)
Requirement already satisfied: pandas>=0.24 in c:\programdata\anaconda3\lib\site-packages (from catboost) (2.2.3)
Requirement already satisfied: scipy in c:\programdata\anaconda3\lib\site-packages (from catboost) (1.15.3)
```

Import Libraries

This cell imports all the necessary Python libraries that will be used throughout the notebook for data manipulation, visualization, model training, and saving.

```
import pandas as pd # Import pandas for data manipulation and analysis
import numpy as np # Import numpy for numerical operations
import matplotlib.pyplot as plt # Import matplotlib for creating static visualizations
import seaborn as sns # Import seaborn for creating statistical graphics

# Import modules from scikit-learn for machine learning tasks
from sklearn.model_selection import train_test_split # For splitting data into training and testing sets
from sklearn.preprocessing import LabelEncoder, StandardScaler # For encoding categorical features and scaling numerical features
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score # For evaluating model performance

# Import various machine learning models
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier # Ensemble models
from xgboost import XGBClassifier # Extreme Gradient Boosting model
from catboost import CatBoostClassifier # CatBoost model

import joblib # For saving and loading Python objects, including trained models
```

Load and Explore the Dataset

This cell loads the accident data from a CSV file into a pandas DataFrame. It then prints the shape of the DataFrame (number of rows and columns), displays the first few rows to show the data structure, and provides information about the columns, including data types and non-null counts.

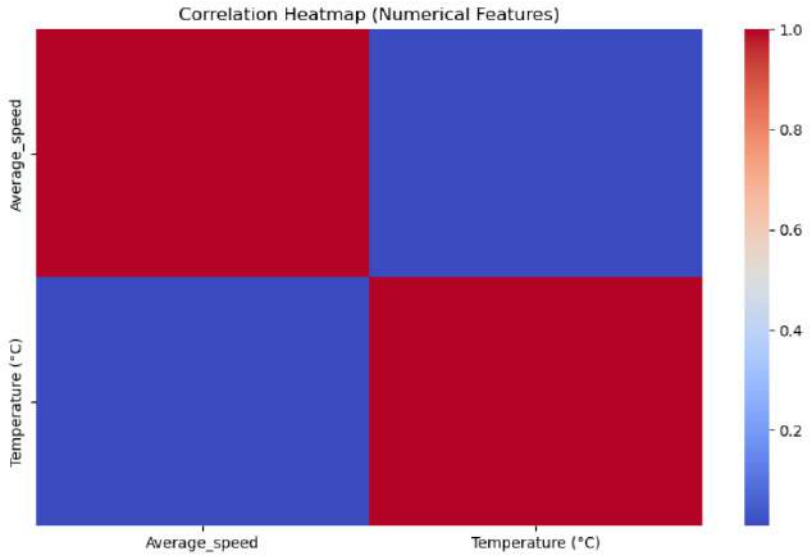
```
# Load the dataset from the specified CSV file into a pandas DataFrame
df = pd.read_csv("RTA_Dataset.csv")

# Print the shape of the DataFrame (number of rows, number of columns)
print("Shape:", df.shape)
# Display the first 5 rows of the DataFrame to get a glimpse of the data
print(df.head())
# Print concise information about the DataFrame, including column data types and non-null values
print(df.info())
```

```
Shape: (15000, 13)
    Time Day_of_week Weather Vehicle_type  Average_speed Driving_Experience  \
0  22:08   Wednesday   Foggy        Truck         120.89            Novice
1  06:33   Wednesday   Clear      Bicycle          97.95      Intermediate
2  01:22    Saturday  Cloudy          Bus         137.44      Intermediate
3  18:02      Friday   Clear          Bus          47.69       Experienced
4  12:45    Thursday   Clear      Bicycle          99.43      Intermediate

  Type_of_Collision     Area Road_Alignment Traffic_Density Road_Condition  \
0       Pedestrian  Highway       Junction            High          Muddy
1       Pedestrian    Rural       Straight             Low            Wet
2  Vehicle overturn    Urban       Straight             Low            Dry
3         Rear-end  Highway       Junction             Low          Muddy
4             Side    Urban       Straight             Low            Dry

   Temperature (°C) Accident_severity
0             18.06      Fatal Injury
1             32.31    Serious Injury
2             16.52      Fatal Injury
3             35.92     Slight Injury
4             33.75    Serious Injury


<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15000 entries, 0 to 14999
Data columns (total 13 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   Time                15000 non-null  object
 1   Day_of_week         15000 non-null  object
 2   Weather             15000 non-null  object
 3   Vehicle_type        15000 non-null  object
 4   Average_speed       15000 non-null  float64
 5   Driving_Experience  15000 non-null  object
 6   Type_of_Collision   15000 non-null  object
 7   Area                15000 non-null  object
 8   Road_Alignment      15000 non-null  object
 9   Traffic_Density     15000 non-null  object
 10  Road_Condition      15000 non-null  object
 11  Temperature (°C)    15000 non-null  float64
 12  Accident_severity   15000 non-null  object
dtypes: float64(2), object(11)
memory usage: 1.5+ MB
None
```

Heatmap

This cell calculates and visualizes the correlation matrix for the numerical features in the dataset using a heatmap. A heatmap helps to understand the relationships between different numerical variables.

```
[25]: # Select only numerical columns for correlation calculation
numerical_df = df.select_dtypes(include=np.number)

# Create a heatmap to visualize the correlation matrix
plt.figure(figsize=(10,6)) # Set the size of the figure
sns.heatmap(numerical_df.corr(), annot=False, cmap="coolwarm") # Generate the heatmap
plt.title("Correlation Heatmap (Numerical Features)") # Set the title of the heatmap
plt.show() # Display the plot
```



Feature Engineering

This cell creates new features from existing ones to improve the model's performance. It extracts the hour from the 'Time' column, creates a 'Day_or_Night' feature, adds an 'Is_Weekend' indicator, and categorizes 'Average_speed' into different bins.

```
# Convert time to hour feature
# Use errors='coerce' to handle potential parsing issues, fillna(0) and astype(int) for robustness
df['Hour'] = pd.to_datetime(df['Time'], format='%H:%M:%S', errors='coerce').dt.hour.fillna(0).astype(int)


# Create new feature: Day_or_Night based on hour
df['Day_or_Night'] = np.where((df['Hour'] >= 6) & (df['Hour'] <= 18), "Day", "Night")

# Create Weekend Feature: 1 if Saturday or Sunday, 0 otherwise
df['Is_Weekend'] = df['Day_of_week'].isin(['Saturday', 'Sunday']).astype(int)

# Speed Category Binning (Ensure bins and labels match the Flask app)
# Categorize 'Average_speed' into 'Low', 'Moderate', and 'High' bins
df['Speed_Category'] = pd.cut(df['Average_speed'], bins=[0,40,80,120], labels=['Low','Moderate','High'], include_lowest=True)

# Print confirmation message
print("\nFeature Engineering Complete")
# Print the names of the new columns added
print("New Columns Added:",
      [col for col in ['Hour', 'Day_or_Night', 'Is_Weekend', 'Speed_Category']
       if col in df.columns])
# Print the list of all columns in the DataFrame after feature engineering
print("\nDataFrame columns after feature engineering:", df.columns.tolist())
```

```
Feature Engineering Complete
New Columns Added: ['Hour', 'Day_or_Night', 'Is_Weekend', 'Speed_Category']

DataFrame columns after feature engineering: ['Time', 'Day_of_week', 'Weather', 'Vehicle_type', 'Average_speed', 'Driving_Experience', 'Type_of_Collision',
'Area', 'Road_Alignment', 'Traffic_Density', 'Road_Condition', 'Temperature (°C)', 'Accident_severity', 'Hour', 'Day_or_Night', 'Is_Weekend', 'Speed_Catego
ry']
```

Check Target Distribution

This cell examines the distribution of the target variable, 'Accident_severity', by counting the occurrences of each severity level. This is important for understanding the class balance in the dataset.

```
target = "Accident_severity"
# Print the distribution of the target variable
print(df[target].value_counts())
```

```
Accident_severity
Fatal Injury      5000
Serious Injury    5000
Slight Injury     5000
Name: count, dtype: int64
```

Handle Missing Values

This cell removes rows with any missing values from the DataFrame. This is a simple way to handle missing data, ensuring that the model doesn't encounter issues with incomplete information.

```
# Remove rows with any missing values from the DataFrame
df = df.dropna()
```

```

```

Identify Categorical and Numerical Columns

This cell separates the DataFrame columns into two lists: one for categorical columns (object and category data types) and one for numerical columns (integer and float data types). This separation is useful for applying different preprocessing steps to each type of column.

```
# Select columns with 'object' or 'category' data types as categorical columns
cat_cols = df.select_dtypes(include='object').columns
# Select columns with 'int64' or 'float64' data types as numerical columns
num_cols = df.select_dtypes(include=['int64','float64']).columns

# Print the identified categorical and numerical column names
print("Categorical:", cat_cols)
print("Numerical:", num_cols)
```

```
Categorical: Index(['Time', 'Day_of_week', 'Weather', 'Vehicle_type', 'Driving_Experience',
       'Type_of_Collision', 'Area', 'Road_Alignment', 'Traffic_Density',
       'Road_Condition', 'Accident_severity', 'Day_or_Night'],
      dtype='object')
Numerical: Index(['Average_speed', 'Temperature (°C)', 'Hour', 'Is_Weekend'], dtype='object')
```

Encode Categorical Features

This cell converts the categorical features into numerical representations using Label Encoding. This is a necessary step as most machine learning algorithms require numerical input. The encoders are saved for later use in the prediction app.

```
# Encode categorical columns
label_encoders = {} # Dictionary to store fitted encoders
for col in df.select_dtypes(include=['object', 'category']).columns:
    le = LabelEncoder() # Initialize LabelEncoder for each column
    df[col] = le.fit_transform(df[col]) # Fit and transform the column
    label_encoders[col] = le # Store the fitted encoder

# Save encoders for later use in the app
joblib.dump(label_encoders, "encoders.pkl")
```

Scale Numerical Features

This cell scales the numerical features using StandardScaler. Scaling helps to standardize the range of numerical features, which can improve the performance of some machine learning algorithms. The scaler is saved for later use in the prediction app.

```
# Separate features (X) and target (y)
X = df.drop('Accident_severity', axis=1)
y = df['Accident_severity']

# Initialize and fit the StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# Save the fitted scaler for later use
joblib.dump(scaler, "scaler.pkl")
```

```
['scaler.pkl']
```

Split Data into Training and Testing Sets

This cell splits the preprocessed data into training and testing sets. The training set is used to train the machine learning models, and the testing set is used to evaluate their performance on unseen data.

```
# Split the scaled data and target variable into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
```

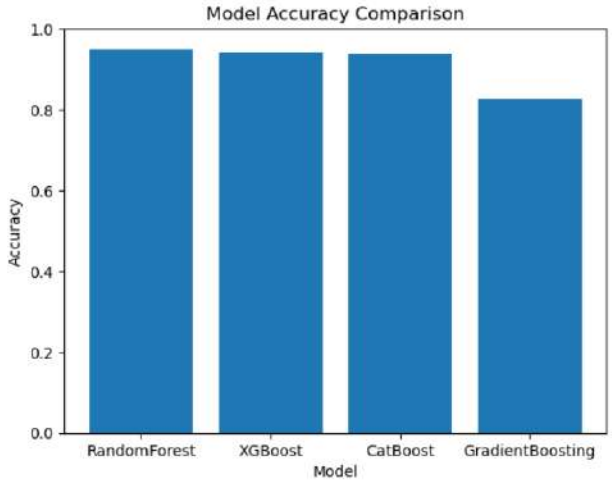
Train and Evaluate Multiple Models

This cell trains several different machine learning models, including RandomForest, XGBoost, CatBoost, and GradientBoosting, on the training data. It then evaluates the performance of each model on the testing data using accuracy as a metric and prints the results.

```python
# Define the models to be trained
models = {
    "RandomForest": RandomForestClassifier(random_state=42),
    "XGBoost": XGBClassifier(eval_metric='mlogloss', random_state=42),
    "CatBoost": CatBoostClassifier(verbose=0), # verbose=0 to suppress training output
    "GradientBoosting": GradientBoostingClassifier()
}

results = {} # Dictionary to store accuracy results for each model

# Iterate through each model, train, predict, and evaluate
for name, model in models.items():
    print(f"Training {name}...")
    model.fit(X_train, y_train) # Train the model
    y_pred = model.predict(X_test) # Make predictions on the test set
    results[name] = accuracy_score(y_test, y_pred) # Calculate and store accuracy
    print(f"{name} Accuracy: {results[name]:.4f}")

# Print the accuracy results for all models
print("\nModel Accuracy Results:")
print(results)
import matplotlib.pyplot as plt

plt.bar(results.keys(), results.values())
plt.title("Model Accuracy Comparison")
plt.xlabel("Model")
plt.ylabel("Accuracy")
plt.ylim(0, 1)
plt.show()
```

```
Training RandomForest...
RandomForest Accuracy: 0.9478
Training XGBoost...
XGBoost Accuracy: 0.9410
Training CatBoost...
CatBoost Accuracy: 0.9380
Training GradientBoosting...
GradientBoosting Accuracy: 0.8284
```

```
Training GradientBoosting...
GradientBoosting Accuracy: 0.8284

Model Accuracy Results:
{'RandomForest': 0.9478032294404807, 'XGBoost': 0.9410439354111904, 'CatBoost': 0.9380398047315058, 'GradientBoosting': 0.8283890349230192}
```

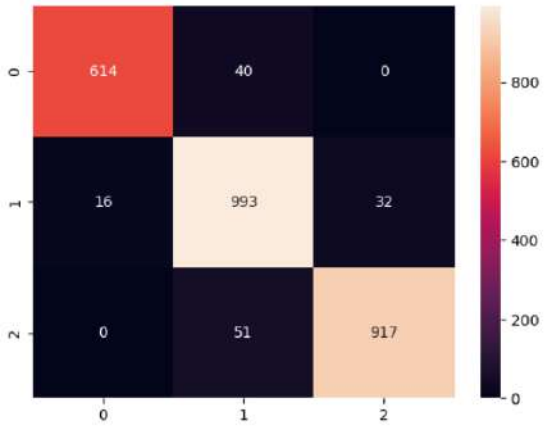


Select and Evaluate the Best Model

This cell identifies the model with the highest accuracy from the trained models. It then prints a detailed classification report and displays a confusion matrix for the best model, providing a comprehensive evaluation of its performance.

```
# Identify the name of the model with the highest accuracy from the results dictionary
best_model_name = max(results, key=results.get)
# Get the best performing model object from the models dictionary
best_model = models[best_model_name]

# Print the name of the best model
print("Best Model:", best_model_name)

# Make predictions on the test set using the best model
y_pred = best_model.predict(X_test)

# Print the classification report to evaluate the model's performance
# The report includes precision, recall, f1-score, and support for each class
print(classification_report(y_test, y_pred))

# Generate and display a confusion matrix as a heatmap
# A confusion matrix shows the counts of true positive, true negative, false positive, and false negative predictions
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d') # annot=True displays the numbers, fmt='d' formats them as integers
plt.show() # Display the plot
```

```
Best Model: RandomForest
              precision    recall  f1-score   support

           0       0.97      0.94      0.96       654
           1       0.92      0.95      0.93      1041
           2       0.97      0.95      0.96       968

    accuracy                           0.95      2663
   macro avg       0.95      0.95      0.95      2663
weighted avg       0.95      0.95      0.95      2663
```

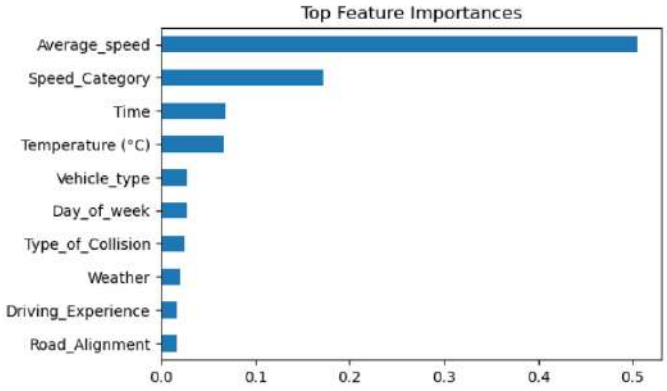


Visualize Feature Importances

This cell visualizes the importance of different features for the trained models (if the model supports feature importance). This helps in understanding which features contribute most to the model's predictions.

```
# Check if the best model has a 'feature_importances_' attribute (common in tree-based models)
if hasattr(best_model, "feature_importances_"):
    # Create a pandas Series from feature importances with feature names as index
    importances = pd.Series(best_model.feature_importances_, index=X.columns)
```

```
# Sort feature importances and select the top 10, then plot as a horizontal bar chart
importances.sort_values().tail(10).plot(kind='barh', figsize=(6,4))
plt.title("Top Feature Importances") # Set the title of the plot
plt.show() # Display the plot
```



Create Templates Folder

This cell creates a folder named `templates`. This folder will be used to store HTML files for the Flask web application.

```
!mkdir template
import os
os.makedirs("template", exist_ok=True)
print("✅ Folder 'templates_frontend' created (or already exists).")
```

```
✅ Folder 'templates_frontend' created (or already exists).
A subdirectory or file template already exists.
```

Create HTML Template for the Web App

This cell writes the HTML code for the web application's user interface into a file named `index.html` inside the `templates` folder. This HTML file contains a form for user input and a section to display the prediction result.

```
%%writefile template/index.html
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>Accident Severity Prediction</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 30px; }
    form { max-width:700px; margin:auto; background:#f8f9fb; padding:20px; border-radius:8px; box-shadow:0 2px 6px rgba(0,0,0,0.08);}
    label { display:block; margin-top:10px; font-weight:600; }
    input, select { width:100%; padding:8px; margin-top:6px; border-radius:6px; border:1px solid #ccc; }
    button { margin-top:14px; padding:12px; width:100%; background:#007bff; color:white; border:none; border-radius:6px; font-size:16px; }
    .result { text-align:center; margin-top:18px; font-size:18px; }
  </style>
</head>
<body>
  <h2 style="text-align:center">Accident Severity Prediction</h2>

  <form method="post" action="/predict">
    <label>Time (HH:MM)</label>
    <input name="Time" type="time" value="14:30">

    <label>Day of Week</label>
    <select name="Day_of_week">
      <option>Monday</option><option>Tuesday</option><option>Wednesday</option>
      <option>Thursday</option><option>Friday</option><option>Saturday</option><option>Sunday</option>
    </select>

    <label>Weather</label>
    <select name="Weather">
      <option>Clear</option><option>Rainy</option><option>Fog</option><option>Cloudy</option><option>Windy</option>
    </select>

    <label>Vehicle Type</label>
    <select name="Vehicle_type">
            <option>Car</option>
            <option>Motorcycle</option>
            <option>Bus</option>
            <option>Truck</option>
            <option>Bicycle</option>
            <option>Auto</option>
          </select>



    <label>Average Speed (km/h)</label>
    <input name="Average_speed" type="number" value="50">

    <label>Driving Experience</label>
    <select name="Driving_Experience">
      <option>Novice</option><option>Intermediate</option><option>Experienced</option>
    </select>
```

Save the Best Model

This cell saves the best-performing trained model to a file using `joblib`. This allows the trained model to be loaded later for making predictions without needing to retrain it.

```
# Save the best-performing trained model to a file
joblib.dump(best_model, f"best_model_{best_model_name}.pkl")
# Print a confirmation message
print("Model Saved Successfully!")
```

```
Model Saved Successfully!
```

Install Dependencies for Flask App

This cell installs the necessary Python libraries to build and run a web application using Flask and expose it to the internet using ngrok.

```
# Install dependencies
import sys
!{sys.executable} -m pip install flask flask-ngrok joblib scikit-learn pyngrok --no-warn-script-location
```

```
    <label>Type of Collision</label>
    <select name="Type_of_Collision">
            <option>Head-on</option>
            <option>Rear-end</option>
            <option>Side</option>
            <option>Vehicle overturn</option>
            <option>Pedestrian</option>
          </select>
```

```
    <label>Area</label>
    <select name="Area">
      <option>Urban</option><option>Rural</option><option>Highway</option>
    </select>

    <label>Road Alignment</label>
    <select name="Road_Alignment">
      <option>Straight</option><option>Curve</option><option>Junction</option>
    </select>

    <label>Traffic Density</label>
    <select name="Traffic_Density">
      <option>Low</option><option>Medium</option><option>High</option>
    </select>

    <label>Road Condition</label>
    <select name="Road_Condition">
      <option>Dry</option><option>Wet</option><option>Muddy</option>
    </select>

    <label>Temperature (°C)</label>
    <input type="number" name="Temperature (°C)" value="25" step="0.1" required>


    <button type="submit">Predict</button>
  </form>
```

```
  {% if prediction %}
    <div class="result"><strong>{{ prediction }}</strong></div>
  {% endif %}
</body>
</html>
```

```
Overwriting template/index.html
```

Run the Flask Web Application with Ngrok

This cell contains the Python code for the Flask web application. It loads the trained model, scaler, and encoders, defines routes for the home page and prediction, processes user input, makes predictions using the loaded model, and serves the results via a web interface exposed through a public ngrok URL.

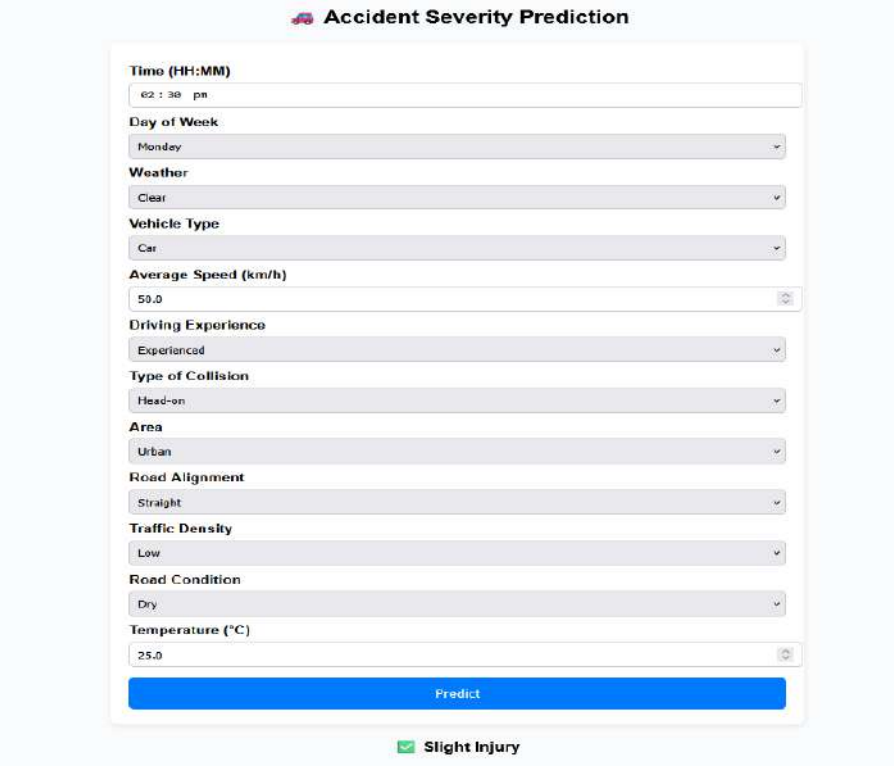
```
# Real-Time Accident Severity Prediction System

# Import necessary libraries for Flask app
from flask import Flask, request, render_template
import pandas as pd
import numpy as np
import joblib
from datetime import datetime

# Flask Setup
app = Flask(__name__, template_folder="templates")

# Load Model, Scaler, and Encoders
# Define file paths for saved model, scaler, and encoders
MODEL_FILE = "best_model_RandomForest.pkl"
SCALER_FILE = "scaler.pkl"
ENCODER_FILE = "encoders.pkl"

# Load pre-trained objects
model = joblib.load(MODEL_FILE)
scaler = joblib.load(SCALER_FILE)
encoders = joblib.load(ENCODER_FILE)
```

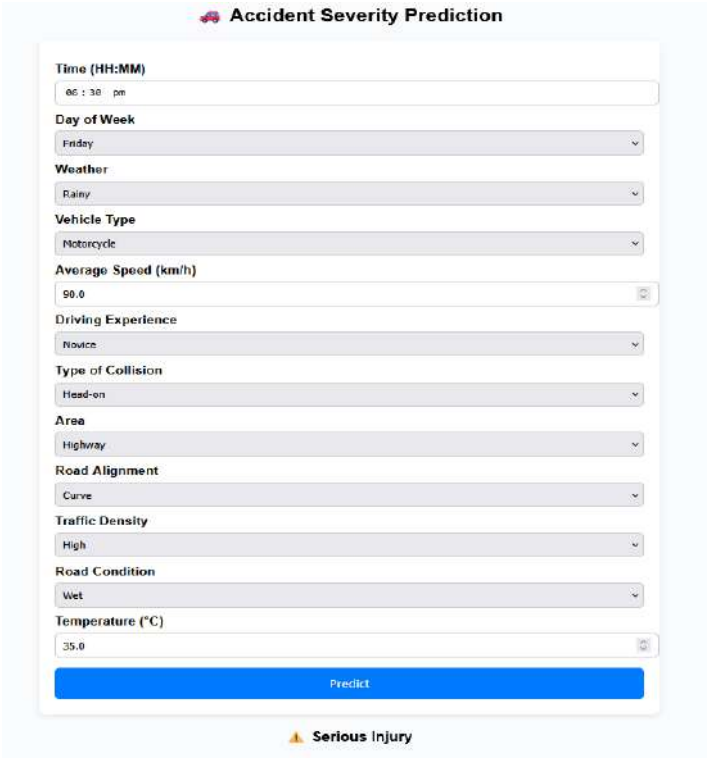
```python
# Define the route for the home page
@app.route('/')
def home():
    return render_template("index.html")

# Prediction route
@app.route('/predict', methods=['POST'])
def predict():
    try:
        # Collect form data
        data = {
            "Time": request.form["Time"],
            "Day_of_week": request.form["Day_of_week"],
            "Weather": request.form["Weather"],
            "Vehicle_type": request.form["Vehicle_type"],
            "Average_speed": float(request.form["Average_speed"]),
            "Driving_Experience": request.form["Driving_Experience"],
            "Type_of_Collision": request.form["Type_of_Collision"],
            "Area": request.form["Area"],
            "Road_Alignment": request.form["Road_Alignment"],
            "Traffic_Density": request.form["Traffic_Density"],
            "Road_Condition": request.form["Road_Condition"],
            "Temperature (°C)": float(request.form["Temperature (°C)"])
        }

        df = pd.DataFrame([data])

        # Feature Engineering
        df["Hour"] = pd.to_datetime(df["Time"], errors='coerce').dt.hour.fillna(0).astype(int)
        df["Day_or_Night"] = np.where((df["Hour"] >= 6) & (df["Hour"] <= 18), "Day", "Night")
        df["Is_Weekend"] = df["Day_of_week"].isin(["Saturday", "Sunday"]).astype(int)
        df["Speed_Category"] = pd.cut(
            df["Average_speed"], bins=[0, 40, 80, 120],
            labels=["Low", "Moderate", "High"], include_lowest=True
        )


        df_input = df.copy()

        # Apply encoders to categorical columns
        for col, encoder in encoders.items():
            if col in df_input.columns:
                try:
                    df_input[col] = encoder.transform(df_input[col])
                except ValueError:
                    df_input[col] = encoder.transform([encoder.classes_[0]])

        # Match feature order
        feature_order = [
            'Time', 'Day_of_week', 'Weather', 'Vehicle_type',
            'Average_speed', 'Driving_Experience', 'Type_of_Collision', 'Area',
            'Road_Alignment', 'Traffic_Density', 'Road_Condition', 'Temperature (°C)',
            'Hour', 'Day_or_Night', 'Is_Weekend', 'Speed_Category'
        ]
        df_input = df_input.reindex(columns=feature_order, fill_value=0)
        df_input = df_input.apply(pd.to_numeric, errors='coerce').fillna(0)

        # Scale and predict
        df_scaled = scaler.transform(df_input)
        pred = model.predict(df_scaled)[0]
```

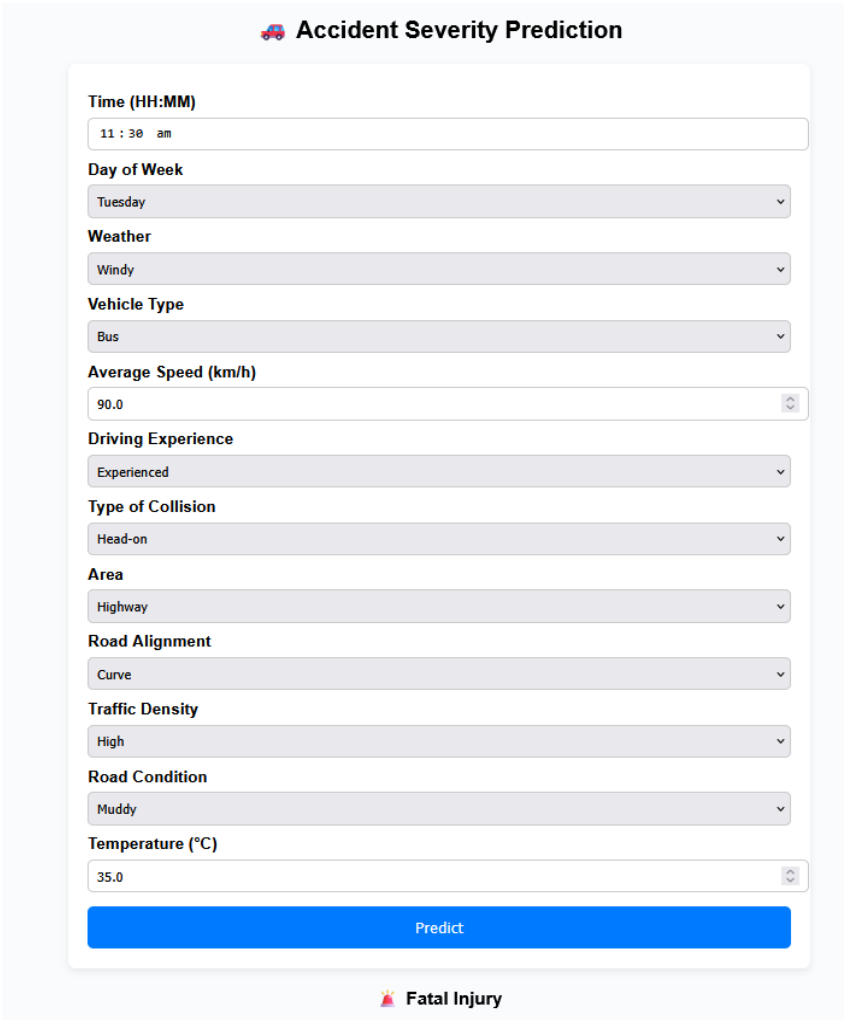
```
        # Decode prediction manually
        severity_mapping = {
            0: "Fatal Injury",
            1: "Serious Injury",
            2: "Slight Injury"
        }
        label = severity_mapping.get(int(pred), "Unknown")

        # Format prediction for display
        if label == "Fatal Injury":
            result = "🚨 Fatal Injury"
        elif label == "Serious Injury":
            result = "⚠️ Serious Injury"
        else:
            result = "✅ Slight Injury"

        # Return result to template
        return render_template("index.html", prediction=result)

    except Exception as e:
        # Handle any errors during prediction and display an error message
        return render_template("index.html", prediction=f"❌ Error: {e}")

# Run Flask app locally

if __name__ == "__main__":
    print("🚀 Flask app running at: http://127.0.0.1:5000/")
    app.run(port=5000, debug=False, use_reloader=False)
```

```
🚀 Flask app running at: http://127.0.0.1:5000/
 * Serving Flask app '__main__'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [11/Nov/2025 15:55:47] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [11/Nov/2025 15:55:55] "POST /predict HTTP/1.1" 200 -
```

The following are some testcases output screens:




Accident Severity Prediction
Time (HH:MM)
06 : 30 pm
Day of Week
Friday
Weather
Rainy
Vehicle Type
Motorcycle
Average Speed (km/h)
90.0
Driving Experience
Novice
Type of Collision
Head-on
Area
Highway
Road Alignment
Curve
Traffic Density
High
Road Condition
Wet
Temperature (°C)
35.0
Predict
Serious Injury


Accident Severity Prediction
Time (HH:MM)
11 : 30 am
Day of Week
Tuesday
Weather
Windy
Vehicle Type
Bus
Average Speed (km/h)
90.0
Driving Experience
Experienced
Type of Collision
Head-on
Area
Highway
Road Alignment
Curve
Traffic Density
High
Road Condition
Muddy
Temperature (°C)
35.0
Predict
Fatal Injury

Functional Test Cases for Flask App

Each test simulates form inputs from your HTML and checks how your model responds.

Test Case	Scenario Description	Input Values	Expected Result or Output
1	Clear weather, experienced driver, moderate speed	Time=02:30pm, Day_of_week=Monday, Weather=Clear, Vehicle_type=Car, Average_speed=50, Driving_Experience=Experienced, Type_of_Collision=Rear-end, Area=Urban, Road_Alignment=Straight, Traffic_Density=Medium, Road_Condition=Dry, Temperature (°C)=28	✅ Slight Injury
2	Rainy weather, novice driver, high speed	Time=6:00pm, Day_of_week=Friday, Weather=Rainy, Vehicle_type=Bike, Average_speed=90, Driving_Experience=Novice, Type_of_Collision=Head-on, Area=Highway, Road_Alignment=Curve, Traffic_Density=High, Road_Condition=Wet, Temperature (°C)=24	⚠️ Serious Injury
3	Foggy morning, rural area, low traffic	Time=06:30am, Day_of_week=Sunday, Weather=Fog, Vehicle_type=Truck, Average_speed=45, Driving_Experience=Experienced, Type_of_Collision=Rear-end, Area=Rural, Road_Alignment=Straight, Traffic_Density=Low, Road_Condition=Dry, Temperature (°C)=18	✅ Slight Injury
4	Night, rainy, sharp junction	Time=10:15pm, Day_of_week=Saturday, Weather=Rainy, Vehicle_type=Car, Average_speed=70, Driving_Experience=Intermediate, Type_of_Collision=Side-impact,	⚠️ Serious Injury

		Area=Urban, Road_Alignment=Junction, Traffic_Density=High, Road_Condition=Wet, Temperature (°C)=22	
5	Highway, high-speed, windy weather	Time=11:00am, Day_of_week=Tuesday, Weather=Windy, Vehicle_type=Bus, Average_speed=100, Driving_Experience=Experienced, Type_of_Collision=Head-on, Area=Highway, Road_Alignment=Curve, Traffic_Density=High, Road_Condition=Dry, Temperature (°C)=35	🚨 Fatal Injury

Negative Test Cases (Validation & Edge Cases)

Test Case	Scenario	Invalid Input	Expected Behavior
6	Missing numeric input	Average_speed left empty	Show error message / form validation warning
7	Invalid temperature input	Temperature (°C) = “abc”	Handle gracefully (show ❌ Error: could not convert string to float)
8	Unseen category	Weather = “Snowy” (not in training set)	Encoded as default / handled with fallback encoder
9	Missing field	“Type_of_Collision” not selected	Should not crash; use default or error message
10	Extreme temperature	Temperature (°C) = 55	Should still predict a severity class

9.3 Instructions Reference

To reproduce the project:

1. Open the notebook and load the dataset.
2. Run preprocessing, feature engineering, and model training cells.
3. Run the Flask app cell and access the generated link.

Note: Please verify that the related links are correctly placed in the required locations.

9.4 Summary Statement

This project presents a complete end-to-end workflow for a Real-Time Accident Severity Prediction System, encompassing data preprocessing, feature engineering, model training, evaluation, and deployment through a Flask web application. The system effectively demonstrates how machine learning can analyze road accident data to predict severity levels with high accuracy, enabling quicker response decisions and enhancing road safety management. It validates both technical robustness and real-world applicability, highlighting the potential of AI-driven predictive analytics in public safety systems.

THE END